

РОЗДІЛ 4. ТЕСТУВАННЯ ТА ДОСЛІДНА ЕКСПЛУАТАЦІЯ

4.1. Методика тестування

Тестування системи NetVigil проводилось за комплексною методикою, що включає чотири рівні:

1. **Функціональне тестування** — перевірка працездатності всіх модулів та API endpoints.
2. **Тестування безпеки** — імітація кібератак для перевірки здатності системи виявляти загрози.
3. **Навантажувальне тестування** — оцінка продуктивності під різним навантаженням.
4. **Користувацьке тестування** — перевірка зручності інтерфейсу та коректності відображення даних.

Тестове середовище включало: - Сервер Ubuntu 22.04 LTS; - Docker 24.x + Docker Compose 2.x; - NVIDIA GPU для прискорення LLM-інференсу; - Браузери Chrome 125, Firefox 126.

4.2. Функціональне тестування

4.2.1. Тестування API Endpoints

[Таблиця 4.1 — Результати функціонального тестування API \(формат Excel/XLSX\)](#)

4.2.2. Тестування фронтенду

Перевірено коректність роботи всіх п'яти сторінок SPA:

- **Dashboard:** KPI-картки оновлюються при завантаженні, графіки коректно відображають дані, кругові індикатори (Memory, Disk) анімовано відображають поточні значення.
- **Log Viewer:** фільтри працюють коректно, підсвітка рівнів логів (info, warning, error, danger) відображається правильно, кнопка «Analyze with AI» активується після завантаження логів.
- **Traffic Analysis:** drag-and-drop завантаження PCAP працює, статистика відображається коректно, графік протоколів будується правильно.

- **Alerts:** алерти відображаються з правильною класифікацією (critical/warning/resolved).
- **AI Chat:** повідомлення відправляються та відповіді отримуються, форматування тексту працює.

4.3. Тестування безпеки

4.3.1. Імітація SQL Injection

Для тестування генеруються HTTP-запити з SQL-ін'єкціями до WordPress:

```
curl "http://localhost/wp-login.php?log=admin'%200R%20'1'='1"  
curl "http://localhost/?p=1%20UNION%20SELECT%20*%20FROM%20wp_users"
```

Результат: AI-адаптер та вебдодаток NetVigil коректно ідентифікують загрозу як «SQL Injection» з risk_score 8-9 та генерують рекомендації щодо блокування IP-адреси.

4.3.2. Імітація Brute-force Attack

Генерування множинних невдалих спроб входу:

```
for i in $(seq 1 50); do  
curl -X POST "http://localhost/wp-login.php" \  
-d "log=admin&pwd=wrong_password_$i"  
done
```

Результат: система визначає загрозу як «Brute-force Attack» з risk_score 7-8, рекомендує блокування через fail2ban або firewall.

4.3.3. Імітація XSS

```
curl "http://localhost/?s=<script>alert('xss')</script>"  
curl "http://localhost/?p=1&comment=<img%20onerror=alert(1)%20src=x>"
```

Результат: LLM коректно класифікує як «Cross-Site Scripting» і рекомендує валідацію та екранування вхідних даних.

4.3.4. Аналіз PCAP з підозрілим трафіком

Завантажено тестовий PCAP-файл з ознаками порт-сканування (100+ TCP SYN-пакетів на різні порти з однієї IP-адреси).

Результат: LLM ідентифікує аномалію як потенційне сканування портів, визначає IP-адресу атакуючого та рекомендує блокування на рівні firewall.

[Таблиця 4.2 — Результати тестування виявлення загроз \(формат Excel/XLSX\)](#)

4.4. Навантажувальне тестування

Тестування проведено за допомогою утиліти wrk:

```
wrk -t4 -c50 -d30s http://localhost:5000/api/dashboard  
wrk -t4 -c50 -d30s http://localhost:5000/api/logs
```

[Таблиця 4.3 — Результати навантажувального тестування \(формат Excel/XLSX\)](#)

LLM-аналіз є найбільш ресурсоємною операцією (12с у середньому). Це обумовлено часом інференсу моделі Llama-3, проте є прийнятним для задач аналізу безпеки.

4.5. Аналіз результатів

На основі проведених тестувань можна зробити наступні висновки:

Функціональність — всі 12 API endpoints працюють коректно, фронтенд коректно відображає дані.

Безпека — система демонструє загальну точність виявлення загроз на рівні 90%. Найвищу точність показує виявлення brute-force атак (100%) та сканування портів (100%). Дещо нижчу точність мають XSS (80%) — це пов'язано з різноманіттям payload-шаблонів XSS.

Продуктивність — API endpoints без LLM відповідають за 200-500мс, що відповідає вимогам ТЗ (до 2 секунд). LLM-аналіз займає 12-45 секунд, що прийнятно для аналітичних задач.

Масштабованість — система підтримує до 50 одночасних з'єднань для API та 10 одночасних LLM-запитів.

4.6. Рекомендації щодо вдосконалення

1. **Fine-tuning LLM** — дотренування моделі на спеціалізованих датасетах кіберзагроз для підвищення точності класифікації.
2. **Кешування результатів** — впровадження Redis для кешування частих запитів до Loki та Prometheus.
3. **WebSocket** — заміна polling-механізму на WebSocket для отримання алертів у реальному часі.
4. **RAG (Retrieval-Augmented Generation)** — підключення бази знань CVE/NVD для збагачення контексту LLM.
5. **RBAC** — додавання системи автентифікації та рольового доступу до вебдодатку.
6. **Інтеграція з SIEM** — підтримка експорту алертів у форматі CEF/LEEF для інтеграції з корпоративними SIEM.